

Rapport de sécurité — OWASP Top 10

Projet : HealthSync — API de gestion de rendez-vous médicaux

Version audité : branche develop au 24/07/2026

Référentiel : OWASP Top 10:2021

1. Contexte et méthodologie

Périmètre

L'audit couvre les deux services applicatifs du dépôt :

- **api/** — service REST (Hono.js), qui expose l'ensemble des endpoints clients : authentification, gestion des patients, des médecins, des spécialités, des plannings et des rendez-vous.
- **grpc-service/** — service gRPC interne, dédié au calcul des créneaux disponibles d'un médecin.

Les données manipulées sont des **données de santé à caractère personnel** (identité des patients, historique de rendez-vous médicaux), ce qui place la barre de sécurité plus haut que pour une application classique : une fuite relève du RGPD renforcé et, en production, imposerait un hébergement certifié HDS.

Qu'est-ce que l'OWASP Top 10 ?

L'**OWASP** (*Open Worldwide Application Security Project*) est une fondation à but non lucratif qui publie des ressources libres sur la sécurité applicative. Son document le plus connu, le **Top 10**, recense les dix catégories de risques les plus critiques pour les applications web. Il est établi à partir de données réelles collectées auprès d'entreprises et de tests d'intrusion, et mis à jour périodiquement — la version en vigueur ici est celle de 2021.

Il ne s'agit pas d'une liste de failles précises, mais de **familles de risques** : chaque catégorie regroupe de nombreuses vulnérabilités partageant une même cause profonde. Le corps de ce rapport se concentre sur l'analyse du code de HealthSync ; une définition de chacune des dix catégories est fournie en **annexe** (section 6) pour le lecteur souhaitant resituer le référentiel.

Méthode

Trois approches complémentaires ont été utilisées :

1. **Revue de code manuelle** — lecture systématique des contrôleurs, services et middlewares, en confrontant chaque catégorie OWASP au code réel plutôt qu'à l'intention supposée.
2. **Analyse des dépendances** — npm audit sur les deux services.
3. **Vérification des en-têtes HTTP** — curl -I sur les endpoints publics.

Convention de statuts

Statut	Signification
Couvert	Une mesure adéquate était déjà en place avant l'audit.
Corrigé	Une faiblesse a été identifiée pendant l'audit et corrigée.
Risque accepté	Une faiblesse est identifiée mais non corrigée ; la justification et l'impact sont documentés.

L'objectif de ce rapport n'est pas d'afficher un score parfait, mais de rendre compte honnêtement de l'état réel du code : les points non traités sont explicitement listés en section 4.

2. Synthèse

#	Catégorie	Statut
A01	Broken Access Control	Corrigé
A02	Cryptographic Failures	Corrigé
A03	Injection	Couvert
A04	Insecure Design	Corrigé
A05	Security Misconfiguration	Corrigé
A06	Vulnerable and Outdated Components	Risque accepté
A07	Identification and Authentication Failures	Couvert
A08	Software and Data Integrity Failures	Couvert
A09	Security Logging and Monitoring Failures	Risque accepté

#	Catégorie	Statut
A10	Server-Side Request Forgery	Non applicable

3. Détail par catégorie

A01 — Broken Access Control · Corrigé

Mesures en place. Le contrôle d'accès repose sur un middleware unique, `auth(allowedRoles?)` (`api/src/middleware/auth.middleware.ts`), qui vérifie la validité du JWT puis, si des rôles sont précisés, que le rôle du porteur figure dans la liste autorisée. Un token absent ou invalide donne un 401, un rôle non autorisé un 403.

Au-delà du rôle, les endpoints de rendez-vous vérifient également la **propriété de la ressource** : un patient ne peut consulter, modifier ou annuler qu'un rendez-vous dont il est le titulaire (`appointments.controller.ts`). Sur `GET /appointments` et `GET /appointments/history`, un patient qui tenterait de passer le `patient_id` d'un tiers en paramètre voit sa demande silencieusement ramenée à son propre identifiant.

Faiblesse identifiée. `GET /appointments/doctor/:doctorId` n'imposait que l'authentification, sans restriction de rôle ni vérification de propriété. Tout utilisateur authentifié — y compris un patient — pouvait donc lister l'intégralité des rendez-vous de n'importe quel médecin, et récupérer au passage les `patient_id` d'autres patients. Il ne s'agissait pas d'une faille d'authentification mais d'un manquement au principe de moindre privilège.

Correctif appliqué. La route est désormais restreinte aux rôles `medecin` et `admin`. Un test vérifie explicitement qu'un patient reçoit un 403, ce qui empêche toute régression silencieuse.

A02 — Cryptographic Failures · Corrigé

Mesures en place. Les mots de passe sont hachés avec **Argon2** (`auth.service.ts`), algorithme recommandé par l'OWASP et vainqueur de la Password Hashing Competition. Aucun mot de passe n'est stocké ni journalisé en clair.

Faiblesses identifiées et corrigées.

1. Le champ `password_hash` était exposé dans les réponses de plusieurs endpoints du module médecins. Bien qu'un hash Argon2 ne soit pas directement exploitable, sa divulgation offre à un attaquant une cible pour une attaque hors ligne et n'a aucune justification fonctionnelle. Le champ n'est désormais jamais sélectionné, quel que soit le contexte d'appel.
2. Le paquet `bcrypt` figurait dans les dépendances sans être utilisé nulle part dans le code — vestige d'un choix initial abandonné au profit d'Argon2. Toute dépendance inutilisée élargit inutilement la surface d'attaque et entretient l'ambiguïté sur l'algorithme réellement employé. Elle a été retirée.

Point de vigilance. Les échanges se font en HTTP en environnement de développement. Une mise en production impose obligatoirement TLS : sans lui, les jetons et les identifiants transitent en clair, et l'en-tête `Strict-Transport-Security` déjà émis par l'application (voir A05) reste sans effet.

A03 — Injection · Couvert

Mesures en place. L'essentiel des accès aux données passe par **Prisma**, qui génère systématiquement des requêtes paramétrées : les valeurs fournies par le client ne sont jamais concaténées dans la requête SQL.

Le service gRPC constitue le seul endroit du projet où du SQL est écrit à la main (`grpc-service/src/availableSlots.service.ts`), pour tirer parti de `generate_series` dans le calcul des créneaux. Cette portion a fait l'objet d'une vérification spécifique : **tous les paramètres variables sont passés via des placeholders** (\$1 à \$4), y compris l'intervalle de durée, et transmis séparément au driver pg.

```
SELECT
  available_slots AS start_time,
  (available_slots + $2::interval) AS end_time
FROM generate_series($3::timestampz, $4::timestampz, $2::interval) AS available_slots
LEFT JOIN appointments a ON (a.doctor_id = $1 AND ...)
```

Aucune concaténation de chaîne n'a été relevée. En complément, toutes les entrées HTTP sont validées par des schémas **Zod** avant d'atteindre la couche métier, ce qui rejette en amont les valeurs de type ou de format inattendus.

A04 — Insecure Design · Corrigé

Faiblesse identifiée. Aucune limitation du nombre de tentatives n'existait sur `POST /auth/login` ni sur `POST /auth/register`. Un attaquant pouvait donc enchaîner des essais de mots de passe

sans contrainte, ou créer massivement des comptes. C'est une faiblesse de conception et non d'implémentation : chaque requête prise isolément était traitée correctement, mais rien n'encadrait leur répétition.

Correctif appliqué. Un middleware de rate limiting a été développé (`api/src/middleware/rateLimit.middleware.ts`).

Stratégie retenue : fenêtre glissante (*sliding window*). Pour chaque client, les horodatages des requêtes récentes sont conservés ; à chaque nouvelle requête, ceux qui sortent de la fenêtre sont écartés du décompte. Ce choix a été préféré à une **fenêtre fixe**, qui présente une faiblesse connue : la remise à zéro périodique permet à un attaquant de concentrer deux quotas complets de part et d'autre de la limite, doublant de fait le nombre de tentatives sur un court intervalle. Un **token bucket** aurait été plus adapté au lissage d'un débit soutenu qu'à la protection contre une rafale d'essais.

Seuils configurés : 5 requêtes par tranche de 15 minutes, par adresse IP. Le seuil est suffisamment strict pour rendre une attaque par force brute impraticable, et suffisamment large pour ne pas pénaliser un utilisateur légitime se trompant plusieurs fois.

Le rejet intervient **avant** le contrôleur : la réponse 429 Too Many Requests, accompagnée d'un en-tête `Retry-After`, est renvoyée sans aucune requête en base ni vérification de mot de passe.

Six tests couvrent le middleware : passage sous le seuil, blocage au-delà, présence du `Retry-After`, indépendance des quotas entre adresses IP, réouverture effective après glissement de la fenêtre, et identification correcte du client derrière une chaîne de proxys.

A05 — Security Misconfiguration · Corrigé

Faiblesse identifiée. L'application n'émettait aucun en-tête HTTP de sécurité.

Correctif appliqué. Le middleware `hono/secure-headers` a été activé globalement. Les en-têtes suivants sont désormais présents sur toutes les réponses, vérifiés par `curl -I` :

En-tête	Valeur	Rôle
<code>Content-Security-Policy</code>	<code>default-src 'self'; ...</code>	Restreint les origines de chargement des ressources
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Empêche le navigateur de réinterpréter le type MIME
<code>X-Frame-Options</code>	<code>SAMEORIGIN</code>	Protège du <i>clickjacking</i>
<code>Strict-Transport-Security</code>	<code>max-age=15552000; includeSubDomains</code>	Impose HTTPS une fois le site visité
<code>Referrer-Policy</code>	<code>no-referrer</code>	Évite la fuite d'URL vers des tiers
<code>Cross-Origin-Resource-Policy</code>	<code>same-origin</code>	Bloque le chargement cross-origin des réponses

Arbitrage sur la CSP. L'interface Swagger servie sur `/docs` charge ses ressources depuis `cdn.jsdelivr.net` et initialise son affichage via un script *inline*, sans mécanisme de nonce ou de hash exposé par la librairie. La directive `script-src` autorise donc `'unsafe-inline'`. Ce relâchement est **assumé et circonscrit** : `/docs` ne sert que de la documentation publique et ne manipule aucune donnée sensible. Une CSP stricte reste applicable si la documentation est retirée de l'environnement de production, ce qui serait la pratique recommandée.

Point positif relevé. La gestion des erreurs est cohérente : les blocs `catch` renvoient des messages génériques sans jamais exposer de trace d'exécution ni de détail interne au client.

A06 — Vulnerable and Outdated Components · Risque accepté

Constat. `npm audit` a été exécuté sur les deux services :

- **grpc-service** — aucune vulnérabilité.
- **api** — une vulnérabilité de sévérité **modérée** sur `@hono/node-server` (versions antérieures à 2.0.5) : une traversée de répertoire via un antislash encodé (`%5C`) sous Windows, affectant le middleware `serve-static`.

Décision : risque accepté, non corrigé. Deux raisons :

1. **La surface vulnérable n'est pas exploitée.** L'application n'utilise `serveStatic` nulle part. Le fichier `swagger.yaml` est servi par une route Hono classique lisant le fichier via `readFileSync`, sans passer par le middleware concerné. Le code vulnérable est présent dans l'arbre de dépendances mais jamais atteint.
2. **Le correctif impose une montée de version majeure**, signalée comme *breaking change*, dont la validation excède le temps disponible sur ce projet.

Recommandation. Planifier la migration vers `@hono/node-server v2` et intégrer `npm audit` à la chaîne d'intégration continue afin que toute nouvelle vulnérabilité soit signalée automatiquement plutôt

que découverte lors d'un audit ponctuel.

A07 — Identification and Authentication Failures · Couvert

Mesures en place.

- **Jetons à durée de vie courte.** L'*access token* JWT expire au bout de 15 minutes, ce qui limite la fenêtre d'exploitation en cas de vol.
- **Rotation du *refresh token*.** À chaque rafraîchissement, l'ancien jeton est révoqué et un nouveau est émis, dans une **transaction** garantissant qu'on ne peut jamais aboutir à un état où l'ancien est révoqué sans que le nouveau existe. Le contrôleur vérifie en outre que le jeton présenté est bien connu en base, non révoqué, et appartient effectivement à l'utilisateur du *payload*.
- **Stockage sécurisé du *refresh token*.** Il est déposé dans un cookie `httpOnly` (inaccessible au JavaScript, donc non exfiltrable via une XSS), `secure` (transmis uniquement en HTTPS) et `sameSite: Strict` (protection CSRF).
- **Pas d'énumération de comptes.** Un email inexistant et un mot de passe erroné renvoient tous deux le même message générique ("Invalid email or password") avec le même code 401. Un attaquant ne peut donc pas déterminer quels emails sont enregistrés.
- **Pas d'élévation de privilèges à l'inscription.** Le rôle est figé côté serveur à patient dans `registerService` et n'est jamais lu depuis le corps de la requête, ce qui neutralise une attaque de type *mass assignment*. Un test vérifie qu'un champ `role` envoyé par le client est bien ignoré.
- **Complexité minimale du mot de passe** imposée par validation Zod à l'inscription.
- Depuis cet audit, le **rate limiting** décrit en A04 protège en complément contre la force brute.

Point de vigilance. Le secret de signature est lu via `process.env.JWT_SECRET!`. L'assertion TypeScript `!` supprime l'avertissement du compilateur mais n'effectue aucune vérification à l'exécution : si la variable d'environnement était absente en production, l'application démarrerait normalement et n'échouerait qu'à la première authentification. Une garde au démarrage, interrompant le lancement si le secret est manquant, est recommandée (principe du *fail fast*).

A08 — Software and Data Integrity Failures · Couvert

Mesures en place. Le fichier `package-lock.json` est versionné pour les deux services, garantissant que les versions exactes des dépendances installées sont reproductibles et ne dérivent pas silencieusement entre deux environnements.

Une chaîne d'intégration continue (GitHub Actions) exécute le linter, le vérificateur de types et l'intégralité de la suite de tests à chaque *pull request*. Le dépôt impose par ailleurs une revue par le binôme avant tout *merge* sur `develop`, et les branches `main` et `develop` sont protégées : aucune modification ne peut y être poussée directement.

Recommandation. L'ajout d'une vérification automatique des dépendances (`npm audit` en CI, voir A06) compléterait ce dispositif.

A09 — Security Logging and Monitoring Failures · Risque accepté

Constat. L'application ne dispose que du *logger* HTTP fourni par Hono, qui trace la méthode, l'URL, le code de statut et la durée de chaque requête. C'est suffisant pour du développement, mais insuffisant en matière de sécurité :

- les événements sensibles ne sont pas distingués du trafic ordinaire — une série d'échecs d'authentification ressemble à n'importe quelle suite de requêtes ;
- aucun identifiant de corrélation ne permet de suivre une requête entre le service REST et le service gRPC ;
- les logs ne sont ni structurés, ni conservés, ni exportés, ce qui rend toute analyse a posteriori impraticable.

Concrètement, une tentative d'intrusion aujourd'hui bloquée par le rate limiting **ne laisserait aucune trace exploitable**.

Décision : lacune assumée, l'observabilité étant hors du périmètre fonctionnel du MVP.

Recommandation. En production, et *a fortiori* sur des données de santé, ce point devient prioritaire : journalisation structurée (JSON) des événements de sécurité — échecs d'authentification, 403, dépassements de quota — avec identifiant de corrélation propagé entre services, et export vers une solution de centralisation permettant l'alerte.

A10 — Server-Side Request Forgery · Non applicable

L'API n'effectue aucune requête sortante vers une URL fournie ou influencée par un utilisateur. La seule communication sortante est l'appel gRPC vers le service interne, dont l'adresse est fixée par

configuration et jamais dérivée d'une entrée client. La surface d'attaque SSRF est donc nulle en l'état.

Cette conclusion serait à réévaluer si une fonctionnalité de type import depuis une URL externe, appel à un service tiers paramétrable ou *webhook* venait à être ajoutée.

4. Vulnérabilités résiduelles

Les points suivants sont connus, non corrigés, et assumés dans le périmètre de ce projet.

Point	Catégorie	Impact	Mesure recommandée en production
Compteur de rate limiting stocké en mémoire	A04	Quota remis à zéro à chaque redémarrage et non partagé entre instances : la protection s'affaiblit dès qu'on déploie plusieurs répliques	Externaliser le compteur dans un store partagé (Redis)
@hono/node-server < 2.0.5	A06	Nul en l'état, la fonctionnalité vulnérable n'étant pas utilisée	Migrer vers la v2 et ajouter npm audit à la CI
Absence de journalisation de sécurité	A09	Aucune détection ni analyse a posteriori possible en cas d'incident	Logs structurés + corrélation inter-services + centralisation
Absence de garde sur JWT_SECRET	A07	Démarrage silencieux dans une configuration non fonctionnelle	Vérification au démarrage, arrêt immédiat si le secret est absent
'unsafe-inline' dans la CSP	A05	Limité : concerne uniquement la page de documentation publique	Retirer /docs de l'environnement de production et durcir la CSP
Communication en HTTP	A02	Critique en production : jetons et identifiants transmis en clair	TLS obligatoire, terminaison au niveau du reverse proxy
Suppression définitive des patients	RGPD	Le droit à l'oubli est techniquement satisfait, mais aucune anonymisation n'est prévue pour conserver l'historique médical agréé	Arbitrer entre suppression et anonymisation avec le métier

5. Conclusion

L'audit a couvert les dix catégories du référentiel OWASP Top 10:2021. Cinq d'entre elles étaient déjà correctement traitées avant l'audit, quatre faiblesses ont été identifiées et corrigées au cours de celui-ci, et deux points restent ouverts et documentés.

Les quatre correctifs apportés sont les suivants :

1. Restriction du contrôle d'accès sur la liste des rendez-vous d'un médecin (A01) ;
2. Suppression d'une fuite de hash de mot de passe et retrait d'une dépendance cryptographique inutilisée (A02) ;
3. Mise en place d'un rate limiting sur les endpoints d'authentification (A04) ;
4. Activation des en-têtes HTTP de sécurité (A05).

Deux de ces correctifs — la fuite du `password_hash` et le défaut de contrôle d'accès — n'ont pas été trouvés par un outil automatisé mais par **relecture manuelle du code lors de la rédaction de la documentation d'API**. Cela confirme qu'une revue attentive reste indispensable : un scanner de dépendances n'aurait signalé ni l'un ni l'autre.

Les deux lacunes assumées les plus significatives concernent la **persistance du rate limiting** et l'**absence de journalisation de sécurité**. Compte tenu de la nature des données traitées, ces deux points devraient être traités en priorité avant toute mise en production réelle, aux côtés du passage en HTTPS et de l'hébergement sur une infrastructure certifiée HDS.

6. Annexe — Les dix catégories du référentiel

Cette annexe définit chacune des catégories du Top 10 2021, indépendamment de leur application à HealthSync. Elle vise à resituer le référentiel pour un lecteur qui ne le pratique pas au quotidien.

A01 — Broken Access Control. Le contrôle d'accès détermine ce qu'un utilisateur authentifié

a le droit de faire. Il est *rompu* lorsqu'un utilisateur peut agir hors de ses permissions : consulter les données d'autrui, accéder à une fonction d'administration, modifier une ressource dont il n'est pas propriétaire — typiquement en changeant un identifiant dans une URL. C'est la catégorie la plus fréquemment rencontrée du Top 10 2021. À retenir : être authentifié ne dit rien de ce qu'on a le droit de faire, ce sont deux vérifications distinctes.

A02 — Cryptographic Failures. Cette catégorie couvre la protection des données sensibles, au repos comme en transit : absence de chiffrement, algorithmes obsolètes, ou divulgation d'éléments qui devraient rester secrets. L'erreur emblématique est le hachage de mots de passe avec une fonction rapide comme MD5 ou SHA-1, qui permet à un attaquant de tester des milliards de combinaisons par seconde. Les algorithmes dédiés au mot de passe (Argon2, bcrypt, scrypt) sont au contraire délibérément lents et coûteux en mémoire.

A03 — Injection. Une injection survient lorsqu'une donnée fournie par l'utilisateur est interprétée comme une *instruction* au lieu d'être traitée comme une simple valeur — le cas classique étant une requête SQL construite par concaténation de chaînes. La parade fondamentale est la requête paramétrée : la structure et les valeurs sont envoyées séparément, si bien que la base ne peut jamais les confondre. La validation des entrées est une seconde ligne de défense utile, mais ne remplace pas le paramétrage.

A04 — Insecure Design. Introduite en 2021, cette catégorie distingue les défauts de *conception* des défauts d'*implémentation*. Un code peut être parfaitement écrit et rester vulnérable parce qu'une protection n'a jamais été prévue — par exemple une page de connexion irréprochable dans son traitement de chaque requête, mais sans aucune limite au nombre de tentatives. Un défaut d'implémentation se corrige par un correctif ponctuel ; un défaut de conception exige d'ajouter un mécanisme qui n'existait pas.

A05 — Security Misconfiguration. La vulnérabilité ne vient ici ni du code ni de la conception, mais de la *configuration* : options de sécurité laissées désactivées, comptes par défaut conservés, messages d'erreur exposant la structure interne. Les en-têtes HTTP de sécurité en relèvent directement : les navigateurs offrent de nombreuses protections, mais elles ne s'activent que si le serveur les demande explicitement. Ne rien configurer revient à s'en priver.

A06 — Vulnerable and Outdated Components. Une application moderne repose sur des dizaines de bibliothèques tierces, dont chacune peut receler une vulnérabilité découverte après son installation. Le risque est insidieux : une dépendance installée puis oubliée continue de vieillir silencieusement. La bonne pratique consiste à surveiller les vulnérabilités publiées et à évaluer si la portion vulnérable est réellement utilisée — une faille dans une fonction jamais appelée n'a pas la même gravité qu'une faille sur un chemin d'exécution actif.

A07 — Identification and Authentication Failures. Là où A01 traite de ce qu'un utilisateur a le droit de faire, cette catégorie traite de la question antérieure : est-il vraiment celui qu'il prétend être ? Elle couvre les faiblesses du mécanisme d'authentification lui-même — mots de passe faibles autorisés, absence de protection contre la force brute, jetons à durée excessive ou mal invalidés. Un cas souvent négligé est l'énumération de comptes : répondre « email inconnu » d'un côté et « mot de passe incorrect » de l'autre révèle involontairement quels comptes existent.

A08 — Software and Data Integrity Failures. Cette catégorie porte sur la confiance accordée à ce qui entre dans l'application sans vérification d'intégrité : dépendances téléchargées, mises à jour automatiques, artefacts produits par la chaîne d'intégration continue. Les attaques dites de *supply chain* — compromettre un paquet populaire pour atteindre toutes les applications qui en dépendent — en relèvent. Verrouiller les versions de dépendances et protéger la chaîne de build sont les contre-mesures principales.

A09 — Security Logging and Monitoring Failures. Cette catégorie ne décrit pas une faille exploitable directement, mais l'incapacité à détecter qu'une attaque a lieu ou a eu lieu — l'OWASP relève que le délai moyen de détection d'une compromission se compte en mois. Les symptômes typiques sont l'absence de trace des événements sensibles, des journaux non conservés ni centralisés, et l'absence d'alerte automatique. Un dispositif de défense sans journalisation bloque, mais ne permet ni de comprendre ce qui s'est passé, ni de réagir.

A10 — Server-Side Request Forgery. Une SSRF se produit lorsqu'une application effectue une requête vers une URL fournie ou influencée par l'utilisateur, sans la valider. L'attaquant détourne alors le serveur pour qu'il émette des requêtes *en son nom* vers des cibles qu'il ne pourrait pas atteindre lui-même : services internes non exposés, bases du réseau privé, ou points de métadonnées cloud. Le serveur devient un relais involontaire à l'intérieur du périmètre protégé. Cette catégorie est entrée au Top 10 en 2021, portée par la généralisation des architectures cloud et microservices.